

Advanced OOP Concepts in Java

A Comprehensive 24-Slide Presentation

Course: CS202 - Programming Systems

Student: Nguyen Quang Hieu - 25125043

Week: Week 6 — Assignment 3

Slide 1: Title & Cover

```
=====
                        ADVANCED OOP IN JAVA
Multiple Inheritance, Functional Interfaces, Inner/Nested
Classes, and Lambda Expressions
=====
```

- **Topic:** Advanced Object-Oriented Programming & Functional Features in Java
 - **Target Audience:** Advanced Software Developers & Computer Science Students
 - **Environment:** Java 8 / Java 17 LTS+
-

Slide 2: Table of Contents

1. Module 1: Multiple Inheritance in Java

- Single Class Inheritance vs. Multiple Interface Implementation
- Default & Static Interface Methods (Java 8+)
- Resolving Diamond Inheritance Conflicts

2. Module 2: Interfaces & Functional Interfaces

- Abstract Classes vs. Interfaces
- `@FunctionalInterface` Annotation & Single Abstract Method (SAM)
- Built-in Functional Interfaces (`Predicate`, `Function`, `Consumer`, `Supplier`)

3. Module 3: Inner Classes

- Non-Static Member Inner Classes & `Outer.this`
- Local Inner Classes
- Anonymous Inner Classes

4. Module 4: Static Nested Classes

- Static Nested Class vs. Member Inner Class
- Encapsulation & Builder Pattern

5. Module 5: Lambda Expressions & Method References

- Syntax & Target Typing
- Variable Capture & Effectively Final Constraint
- Method References (`::` syntax)

6. Module 6: Modern Java Bonus Concepts

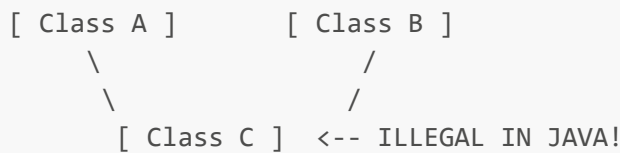
- Sealed Classes & Records (Java 17+)

7. Summary & Key Takeaways

Slide 3: Module 1 — Multiple Inheritance in Java

Why Java Prohibits Multiple Class Inheritance

Unlike C++, Java **does not allow** a class to extend more than one class (`extends A, B` is illegal).



Reasons for Prohibiting Multiple Class Inheritance:

1. **The Diamond Problem:** Prevents ambiguity when both superclasses define state (fields) or method implementations with identical signatures.
2. **Complexity Reduction:** Avoids complex memory layouts, object construction chains, and virtual method table overhead.

Slide 4: Multiple Interface Inheritance & Default Methods

Interface-Based Multiple Inheritance

Java allows a class to implement multiple interfaces (`implements InterfaceA, InterfaceB`).

Starting in **Java 8**, interfaces can contain concrete method implementations using the `default` keyword.

```
interface Printable {
    default void printInfo() {
        System.out.println("Printing info...");
    }
}

interface Loggable {
    default void logInfo() {
        System.out.println("Logging info...");
    }
}

class Document implements Printable, Loggable {
```

```
// Inherits both default methods automatically!  
}
```

Slide 5: Resolving the Diamond Conflict in Java Default Methods

What happens when two implemented interfaces have identical default methods?

```
interface InterfaceA {  
    default void show() { System.out.println("A"); }  
}  
  
interface InterfaceB {  
    default void show() { System.out.println("B"); }  
}  
  
// COMPILER ERROR if show() is not explicitly overridden!  
class Child implements InterfaceA, InterfaceB {  
    @Override  
    public void show() {  
        // Explicitly select which interface implementation to call:  
        InterfaceA.super.show();  
    }  
}
```

Conflict Resolution Rule:

1. **Classes Win:** A method in a superclass always takes precedence over interface default methods.
2. **Explicit Override:** If two interfaces conflict without a superclass implementation, the implementing class **must** override the method and explicitly resolve it (`InterfaceName.super.method()`).

Slide 6: Static Methods in Java Interfaces

Starting in Java 8, interfaces can also declare `static` utility methods:

```
public interface Validator {  
    boolean validate(String data);  
  
    // Static helper method inside interface  
    static boolean isNotNullOrEmpty(String str) {  
        return str != null && !str.trim().isEmpty();  
    }  
}
```

```
// Usage:  
boolean valid = Validator.isNotNullOrEmpty("Hello");
```

- **Key Characteristic:** Interface `static` methods are **not inherited** by implementing classes or sub-interfaces. They must be invoked directly using the interface name (`InterfaceName.method()`).

Slide 7: Module 2 — Interfaces & Functional Interfaces

Abstract Classes vs. Interfaces in Java

Feature	Abstract Class (<code>abstract class</code>)	Interface (<code>interface</code>)
Inheritance	Single (<code>extends</code>)	Multiple (<code>implements</code>)
State / Fields	Can have instance fields (<code>private int x;</code>)	Only <code>public static final</code> constants
Constructors	Can define constructors	Cannot define constructors
Access Modifiers	<code>public</code> , <code>protected</code> , <code>private</code> methods	Default <code>public</code> (private methods added in Java 9)
Primary Intent	"Is-A" identity relationship	"Can-Do" behavioral contract

Slide 8: `@FunctionalInterface` & Single Abstract Method (SAM)

What is a Functional Interface?

A **Functional Interface** is an interface that contains **exactly one abstract method** (SAM). It serves as the target type for **Lambda Expressions** and **Method References**.

```
@FunctionalInterface  
public interface Calculator {  
    int compute(int a, int b); // Single Abstract Method (SAM)  
  
    // Default & static methods do NOT break the SAM requirement:  
    default void logResult(int res) {  
        System.out.println("Result: " + res);  
    }  
}
```

- `@FunctionalInterface` annotation is optional but recommended; it causes the compiler to raise an error if more than one abstract method is added.

Slide 9: Standard Built-in Functional Interfaces (`java.util.function`)

Java provides 4 core built-in functional interfaces:

```
import java.util.function.*;

public class BuiltInFunctionalDemo {
    public static void main(String[] args) {
        // 1. Predicate<T>: Accepts T, returns boolean
        Predicate<Integer> isEven = n -> n % 2 == 0;

        // 2. Function<T, R>: Accepts T, returns R
        Function<String, Integer> strLength = str -> str.length();

        // 3. Consumer<T>: Accepts T, returns void
        Consumer<String> printer = msg -> System.out.println("LOG: " + msg);

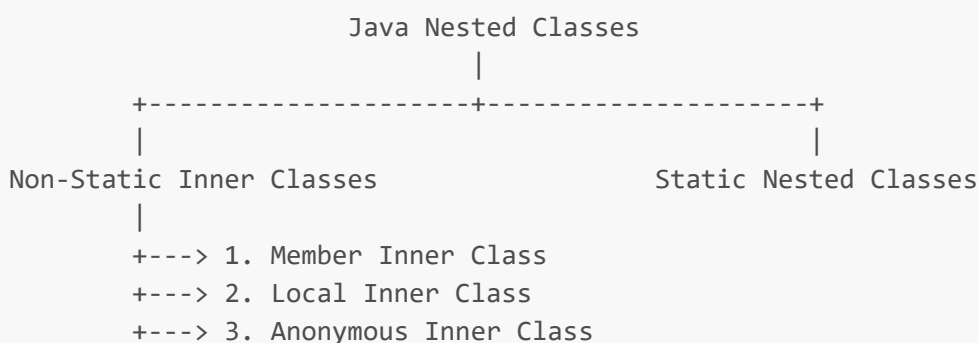
        // 4. Supplier<T>: Accepts nothing, returns T
        Supplier<Double> randomSupplier = () -> Math.random();

        System.out.println(isEven.test(4));           // true
        System.out.println(strLength.apply("Java")); // 4
        printer.accept("System initialized");         // LOG: System initialized
        System.out.println(randomSupplier.get());     // Random double
    }
}
```

Slide 10: Module 3 — Inner Classes in Java

Types of Nested Classes in Java

Java categorizes nested classes into 4 distinct types:



Slide 11: Non-Static Member Inner Class & Outer.this

A member inner class is associated with an **instance** of the enclosing outer class.

```
public class BankAccount {
    private String accountNumber;
    private double balance;

    public BankAccount(String accNum, double initialBalance) {
        this.accountNumber = accNum;
        this.balance = initialBalance;
    }

    // Member Inner Class
    public class Transaction {
        public void deposit(double amount) {
            // Direct access to private fields of outer instance!
            balance += amount;
            System.out.println("Account " + accountNumber + " new balance: $" +
balance);
        }

        public BankAccount getOuterAccount() {
            return BankAccount.this; // Explicit reference to outer instance
        }
    }
}

// Instantiation syntax:
BankAccount acc = new BankAccount("ACC-101", 500.0);
BankAccount.Transaction tx = acc.new Transaction(); // Requires outer instance!
tx.deposit(200.0);
```

Slide 12: Local Inner Classes

A local inner class is defined inside a method body or block.

```
public class PaymentProcessor {
    public void processPayment(double amount) {
        final double feeRate = 0.02; // Local variable

        // Local Inner Class defined inside method
        class Helper {
            void calculateTotal() {
                double total = amount + (amount * feeRate);
            }
        }
    }
}
```

```
        System.out.println("Total charged: $" + total);
    }
}

Helper helper = new Helper();
helper.calculateTotal();
}
}
```

- **Scope Restriction:** Accessible **only** within the method where it is declared.

Slide 13: Anonymous Inner Classes

An inner class without a class name, declared and instantiated simultaneously.

```
public class ThreadDemo {
    public static void main(String[] args) {
        // Anonymous Inner Class implementing Runnable
        Thread t = new Thread(new Runnable() {
            @Override
            public void run() {
                System.out.println("Thread running from anonymous inner class!");
            }
        });
        t.start();
    }
}
```

- **Use Case:** Used heavily in event handling (GUI listeners) and legacy thread creation prior to Java 8 lambdas.

Slide 14: Module 4 — Static Nested Classes in Java

Static Nested Class vs. Member Inner Class

A **static** nested class is associated with the **outer class itself**, NOT an instance of the outer class.

```
public class OuterClass {
    private static String outerStaticField = "Static Data";
    private String outerInstanceField = "Instance Data";

    // Static Nested Class
    public static class StaticNested {
        public void display() {
            System.out.println(outerStaticField); // OK: Accesses outer static
        }
    }
}
```

```
field
    // System.out.println(outerInstanceField); // COMPILE ERROR! Cannot
    // access instance fields!
}
}
}

// Instantiation syntax (does NOT require an outer object!):
OuterClass.StaticNested nestedObj = new OuterClass.StaticNested();
nestedObj.display();
```

Slide 15: Static Nested Classes & The Builder Pattern

Static nested classes are widely used to implement the **Builder Pattern** for immutability:

```
public class UserProfile {
    private final String username; // Required
    private final String email;    // Optional
    private final int age;         // Optional

    private UserProfile(Builder builder) {
        this.username = builder.username;
        this.email = builder.email;
        this.age = builder.age;
    }

    public static class Builder {
        private final String username;
        private String email = "";
        private int age = 0;

        public Builder(String username) { this.username = username; }
        public Builder setEmail(String email) { this.email = email; return this; }
        public Builder setAge(int age) { this.age = age; return this; }

        public UserProfile build() { return new UserProfile(this); }
    }
}

// Usage:
UserProfile user = new UserProfile.Builder("hieung")
    .setEmail("hieu@example.com")
    .setAge(21)
    .build();
```

Slide 16: Inner Class vs. Static Nested Class Summary Matrix

Feature	Member Inner Class	Static Nested Class
Modifier	Non-static	<code>static</code>
Outer Instance Link	Holds implicit reference (<code>Outer.this</code>)	No link to outer instance
Access to Outer Instance Fields	Direct access to all private instance fields	Cannot access outer instance fields
Instantiation Syntax	<code>outerObj.new Inner()</code>	<code>new Outer.StaticNested()</code>
Memory Footprint	Slightly higher (pointer to outer instance)	Lower (standalone class in namespace)

Slide 17: Module 5 — Lambda Expressions in Java

What is a Lambda Expression in Java?

Introduced in Java 8, a lambda expression provides a clear and concise way to implement a **Single Abstract Method (SAM)** interface.

Syntax:

`(parameters) -> { body }`

```
// 1. Anonymous Inner Class (Verbose)
Comparator<String> compOld = new Comparator<String>() {
    @Override
    public int compare(String s1, String s2) {
        return s1.compareTo(s2);
    }
};

// 2. Lambda Expression (Concise!)
Comparator<String> compLambda = (s1, s2) -> s1.compareTo(s2);
```

Slide 18: Target Typing & Type Inference in Java Lambdas

Java infers lambda parameter types from the context (**Target Type**):

```
public class LambdaInferenceDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Charlie", "Alice", "Bob");

        // Type inference: Java infers s1 and s2 are Strings from List<String>
        names.sort((s1, s2) -> s1.compareTo(s2));

        // Explicit types can also be provided if needed:
        names.sort((String s1, String s2) -> s1.compareTo(s2));

        names.forEach(name -> System.out.println(name));
    }
}
```

Slide 19: Variable Capture & "Effectively Final" Rule

Capturing Local Variables in Lambdas

A lambda expression can access local variables from its enclosing scope **only if** those variables are **final** or **effectively final** (never modified after assignment).

```
public void testVariableCapture() {
    int factor = 10; // Effectively final

    Function<Integer, Integer> multiplier = n -> n * factor; // Legal!

    // factor = 20;
    // COMPILE ERROR: Local variable factor defined in an enclosing scope
    // must be final or effectively final!
}
```

Why this rule exists:

Lambdas capture **values**, not variable references, preventing concurrency issues when execution happens asynchronously across threads.

Slide 20: Method References (:: Syntax) in Java

Method references provide an even shorthand syntax for lambdas that simply call an existing method.

```
List<String> items = Arrays.asList("apple", "banana", "cherry");

// 1. Static Method Reference (ContainingClass::staticMethod)
```

```
// Lambda: str -> Integer.parseInt(str)
Function<String, Integer> parser = Integer::parseInt;

// 2. Instance Method Reference of a Particular Object (object::instanceMethod)
// Lambda: msg -> System.out.println(msg)
Consumer<String> printer = System.out::println;

// 3. Instance Method Reference of an Arbitrary Object of a Type
(ContainingType::method)
// Lambda: (s1, s2) -> s1.compareToIgnoreCase(s2)
Comparator<String> comp = String::compareToIgnoreCase;

// 4. Constructor Reference (ClassName::new)
// Lambda: () -> new ArrayList<>()
Supplier<List<String>> listSupplier = ArrayList::new;
```

Slide 21: Module 6 — Modern Java Bonus Concepts (Java 17 LTS)

1. Sealed Classes (**sealed** & **permits**)

Restricts which subclasses are permitted to extend a class, providing strict domain modeling.

```
public sealed class Shape permits Circle, Rectangle, Triangle {
    // Only Circle, Rectangle, Triangle can extend Shape!
}

public final class Circle extends Shape {
    private double radius;
}
```

Slide 22: Records (**record** in Java 14+)

Records eliminate boilerplate for immutable data carrier classes:

```
// Replaces 50+ lines of getters, equals(), hashCode(), toString(), and
constructors!
public record StudentRecord(String id, String name, double gpa) {
    // Custom validation can be added to compact constructor:
    public StudentRecord {
        if (gpa < 0.0 || gpa > 4.0) {
            throw new IllegalArgumentException("Invalid GPA");
        }
    }
}
```

```
}

// Usage:
StudentRecord s = new StudentRecord("25125043", "Hieu Nguyen", 3.9);
System.out.println(s.name()); // "Hieu Nguyen"
```

Slide 23: Feature Comparison Matrix (Python vs. Java)

Feature	Python	Java
Multiple Inheritance	Direct supported for classes via MRO	Disallowed for classes; Supported via Interface default methods
Interface Mechanics	<code>abc.ABC</code> & <code>typing.Protocol</code>	<code>interface</code> keyword & Functional Interfaces
Inner Class Linkage	Manual (<code>self.outer = outer</code>)	Automatic implicit reference (<code>Outer.this</code>)
Static Nested Class	Standard class inside class scope	<code>static class</code> keyword
Lambda Scope Capture	Late binding (lookup at execution time)	Effectively final constraint (value capture)

Slide 24: Conclusion & Key Takeaways

1. **Multiple Inheritance in Java** is safely achieved via **Interfaces with Default Methods** (Java 8+), avoiding diamond class conflicts through strict compiler checks.
2. **Functional Interfaces** (`@FunctionalInterface`) and **Lambda Expressions** transformed Java into a hybrid Object-Oriented and Functional language.
3. **Inner Classes** provide powerful access to outer instance state, whereas **Static Nested Classes** cleanly decouple helper classes and implement patterns like the Builder pattern.
4. **Modern Java Features** (Sealed classes, Records, Method references `::`) continue to refine OOP design while reducing boilerplate code.

End of Presentation — Assignment 3 (Java)